

Experiment 1 :

Aim :

To construct an **AVL Tree** for a given set of elements, implement **Insert** and **Delete** operations, and display the elements of the tree using **In-order Traversal**

Algorithm :

Node structure (assumed)

1. Each node contains: data, left, right, height.
2. height of a NULL node = 0. For a real node: $\text{height} = 1 + \max(\text{height}(\text{left}), \text{height}(\text{right}))$.

Helper functions (step-by-step)

getHeight(node)

1. If node == NULL return 0.
2. Else return node.height.

getBalance(node)

1. If node == NULL return 0.
2. Return $\text{getHeight}(\text{node}.\text{left}) - \text{getHeight}(\text{node}.\text{right})$.

RightRotate(y) — rotate around y (returns new root x)

1. Let $x = y.\text{left}$.
2. Let $T2 = x.\text{right}$.
3. Set $x.\text{right} = y$.
4. Set $y.\text{left} = T2$.
5. Update $y.\text{height} = 1 + \max(\text{getHeight}(y.\text{left}), \text{getHeight}(y.\text{right}))$.
6. Update $x.\text{height} = 1 + \max(\text{getHeight}(x.\text{left}), \text{getHeight}(x.\text{right}))$.
7. Return x (new root of this subtree).

LeftRotate(x) — rotate around x (returns new root y)

1. Let $y = x.\text{right}$.
2. Let $T2 = y.\text{left}$.
3. Set $y.\text{left} = x$.
4. Set $x.\text{right} = T2$.
5. Update $x.\text{height} = 1 + \max(\text{getHeight}(x.\text{left}), \text{getHeight}(x.\text{right}))$.
6. Update $y.\text{height} = 1 + \max(\text{getHeight}(y.\text{left}), \text{getHeight}(y.\text{right}))$.

7. Return y (new root of this subtree).
-

Insert(key) — step-by-step (recursive)

1. If root == NULL: create new node with data = key, height = 1, return it.
 2. If key < root.data: root.left = Insert(root.left, key).
 3. Else if key > root.data: root.right = Insert(root.right, key).
 4. Else (duplicate): return root (no insertion).
 5. Update root.height = 1 + max(getHeight(root.left), getHeight(root.right)).
 6. Compute balance = getBalance(root).
 7. Rebalance using the four cases:
 - **LL case:** balance > 1 and key < root.left.data → return RightRotate(root).
 - **LR case:** balance > 1 and key > root.left.data →
 1. root.left = LeftRotate(root.left),
 2. return RightRotate(root).
 - **RR case:** balance < -1 and key > root.right.data → return LeftRotate(root).
 - **RL case:** balance < -1 and key < root.right.data →
 1. root.right = RightRotate(root.right),
 2. return LeftRotate(root).
 8. If no rotation needed, return root.
-

Delete(key) — step-by-step (recursive)

1. If root == NULL: return root (key not found).
2. If key < root.data: root.left = Delete(root.left, key).
3. Else if key > root.data: root.right = Delete(root.right, key).
4. Else (found node to delete):
 - If root.left == NULL:
 1. Temp = root.right.
 2. Free root (or discard), return temp.
 - Else if root.right == NULL:
 1. Temp = root.left.
 2. Free root, return temp.

- Else (two children):
 1. Find temp = minValueNode(root.right) (leftmost node of right subtree).
 2. Set root.data = temp.data.
 3. root.right = Delete(root.right, temp.data) (delete successor).
- 5. After normal BST deletion, if root == NULL return root.
- 6. Update root.height = 1 + max(getHeight(root.left), getHeight(root.right)).
- 7. Compute balance = getBalance(root).
- 8. Rebalance using the four cases (note: now use child's balances to distinguish LR/RL):
 - **LL case:** balance > 1 and getBalance(root.left) >= 0 → return RightRotate(root).
 - **LR case:** balance > 1 and getBalance(root.left) < 0 →
 1. root.left = LeftRotate(root.left),
 2. return RightRotate(root).
 - **RR case:** balance < -1 and getBalance(root.right) <= 0 → return LeftRotate(root).
 - **RL case:** balance < -1 and getBalance(root.right) > 0 →
 1. root.right = RightRotate(root.right),
 2. return LeftRotate(root).
- 9. Return root.

minValueNode(node) — find inorder successor (leftmost)

1. Start at node.
2. While node.left != NULL, node = node.left.
3. Return node.

Construct AVL tree from a list (step-by-step)

1. Let root = NULL.
2. For each element k in the input list (in given order):
 - root = Insert(root, k).
3. After all inserts, root is the AVL tree root.

In-order Traversal (to display elements)

1. InOrder(node):

- If node == NULL return.
 - InOrder(node.left).
 - Print node.data.
 - InOrder(node.right).
2. Calling InOrder(root) prints elements in sorted order.

Small example (quick walk)

Insert sequence: [30, 20, 10]

1. Insert 30 → tree root 30.
2. Insert 20 → 20 as left child of 30. Heights update → root balance = 1 (OK).
3. Insert 10 → goes left of 20. Now root 30 has balance = 2 (left heavy) and key < root.left.data → **LL case**.
4. Apply RightRotate(30):
 - New root becomes 20, left child 10, right child 30. Tree balanced.

Code :

```
#include <stdio.h>
#include <stdlib.h>
typedef struct node{
    int data,height;
    struct node *left,*right;
}Node;

int height(Node *N){
    if (N == NULL)
        return 0;
    return N->height;
}

Node *newNode(int data){
    Node *node = (Node *) malloc(sizeof(Node));
    node->data = data;
    node->left = NULL;
    node->right = NULL;
    node->height = 1;
    return (node);
}
```

```

Node *rightRotate(Node *y){
    Node *x = y->left;
    Node *T2 = x->right;
    x->right = y;
    y->left = T2;
    y->height = max(height(y->left), height(y->right)) + 1;
    x->height = max(height(x->left), height(x->right)) + 1;
    return x;
}

```

```

Node *leftRotate(Node *x){
    Node *y = x->right;
    Node *T2 = y->left;
    y->left = x;
    x->right = T2;
    x->height = max(height(x->left), height(x->right)) + 1;
    y->height = max(height(y->left), height(y->right)) + 1;
    return y;
}

```

```

int getBalance(Node *N){
    if (N == NULL)
        return 0;
    return height(N->left) - height(N->right);
}

```

```

Node *insertNode(Node *node, int data){
    int balance;

    if (node == NULL)
        return (newNode(data));
    if (data < node->data)
        node->left = insertNode(node->left, data);
    else if (data > node->data)
        node->right = insertNode(node->right, data);
    else
        return node;

    node->height = 1 + max(height(node->left), height(node->right));
    balance = getBalance(node);

    if (balance > 1 && data < node->left->data)
        return rightRotate(node);
}

```

```

if (balance < -1 && data > node->right->data)
    return leftRotate(node);
if (balance > 1 && data > node->left->data) {
    node->left = leftRotate(node->left);
    return rightRotate(node);
}
if (balance < -1 && data < node->right->data) {
    node->right = rightRotate(node->right);
    return leftRotate(node);
}
return node;
}

```

```

Node *minValueNode(Node *node){
    Node *current = node;
    while (current->left != NULL)
        current = current->left;
    return current;
}

```

```

Node *deleteNode(Node *root, int data){
    int balance;

    if (root == NULL)
        return root;
    if (data < root->data)
        root->left = deleteNode(root->left, data);
    else if (data > root->data)
        root->right = deleteNode(root->right, data);
    else {
        if ((root->left == NULL) || (root->right == NULL)) {
            Node *temp = root->left ? root->left : root->right;
            if (temp == NULL) {
                temp = root;
                root = NULL;
            } else
                *root = *temp;
            free(temp);
        } else {
            Node *temp = minValueNode(root->right);
            root->data = temp->data;
            root->right = deleteNode(root->right, temp->data);
        }
    }
}

```

```

if (root == NULL)
    return root;
root->height = 1 + max(height(root->left),
                      height(root->right));
balance = getBalance(root);
if (balance > 1 && getBalance(root->left) >= 0)
    return rightRotate(root);
if (balance > 1 && getBalance(root->left) < 0) {
    root->left = leftRotate(root->left);
    return rightRotate(root);
}
if (balance < -1 && getBalance(root->right) <= 0)
    return leftRotate(root);
if (balance < -1 && getBalance(root->right) > 0) {
    root->right = rightRotate(root->right);
    return leftRotate(root);
}
return root;
}

```

// Print the tree

```

void printTree(Node *root){
    if (root != NULL) {
        printTree(root->left);
        printf("%d ", root->data);
        printTree(root->right);
    }
}

```

```

void main(){
    Node *root = NULL;
    int a[]={22,14,72,44,25,63,98},i;
    clrscr();
    for(i=0;i<sizeof(a)/2;i++)
        root = insertNode(root, a[i]);
    printf("AVL Tree: ");
    printTree(root);
    root = deleteNode(root, 25);
    printf("\nAfter deletion of 25: ");
    printTree(root);
    getch();
}

```

Output :

AVL Tree: 14 22 25 44 63 72 98

After deletion of 25 : 14 22 44 63 72 98

Experiment 2:

AIM

To construct a **B-Tree of order 5** using a given set of 10 random elements stored in an array, and to implement **Searching**, **Insertion**, and **Deletion** operations on the B-Tree.

Key Notes for Order 5

- **Order (m) = 5** means:
 - Each node can have **maximum 4 keys** and **maximum 5 children**.
 - Each node (except root) must have **minimum $\lceil m/2 \rceil - 1 = 2$ keys**.
 - Root can have **minimum 1 key**.
-

Algorithm 1: Construct B-Tree from an Array

1. Start with an **empty B-Tree**.
 2. For each element in the given array:
 1. Insert the element into the B-Tree using the **B-Tree Insertion Algorithm**.
 3. After all insertions, the final tree contains all elements in sorted order.
-

Algorithm 2: Search in a B-Tree

BTree_Search(node, key)

1. Let $i = 0$.
2. While $i < \text{node.n}$ and $\text{key} > \text{node.keys}[i]$, increment i .
3. If $i < \text{node.n}$ and $\text{key} == \text{node.keys}[i]$:
 - **Return node** (key found).
4. Else if $\text{node.isLeaf} == \text{true}$:
 - Return **NULL** (key not found).
5. Else:

- Recursively call `BTree_Search(node.child[i], key)`.
-

Algorithm 3: Insertion in a B-Tree (Order 5)

BTree_Insert(root, key)

1. If root is full ($n == m-1$ keys):
 1. Create a new node `s` and make it the new root.
 2. Set `s.isLeaf = false` and `s.child[0] = root`.
 3. **SplitChild(s, 0)** to split the full root into two nodes.
 4. Call **InsertNonFull(s, key)**.
 2. Else:
 - Call **InsertNonFull(root, key)**.
-

InsertNonFull(node, key)

1. Let $i = \text{node.n} - 1$.
 2. If `node.isLeaf == true`:
 1. While $i \geq 0$ and $\text{key} < \text{node.keys}[i]$:
 - `node.keys[i+1] = node.keys[i]`, decrement i .
 2. Place key at position $i+1$.
 3. Increment `node.n`.
 3. Else (internal node):
 1. While $i \geq 0$ and $\text{key} < \text{node.keys}[i]$, decrement i .
 2. Increment i to get child index.
 3. If `node.child[i]` is full:
 - **SplitChild(node, i)**.
 - If $\text{key} > \text{node.keys}[i]$, increment i .
 4. Call **InsertNonFull(node.child[i], key)**.
-

SplitChild(parent, index)

1. Let $y = \text{parent.child}[\text{index}]$ (full node).
2. Create new node z (same leaf status as y).

3. Set $z.n = \lceil m/2 \rceil - 1$ ($= 2$ for $m=5$).
 4. Copy last 2 keys of y to z .
 5. If y is not leaf, copy last 3 children to z .
 6. Reduce $y.n$ to 2.
 7. Shift children of parent to make space for z .
 8. Shift keys of parent to make space for middle key.
 9. Move middle key of y to parent's $keys[index]$.
 10. Insert z into $parent.child[index+1]$.
 11. Increment $parent.n$.
-

Algorithm 4: Deletion in a B-Tree (Order 5)

BTree_Delete(node, key)

1. Find idx such that $key \leq node.keys[idx]$ or $idx == node.n$.
 2. If $idx < node.n$ and $node.keys[idx] == key$:
 - **Case 1:** Node is leaf $\rightarrow$ Remove key directly.
 - **Case 2:** Node is internal:
 1. If $child[idx]$ has ≥ 2 keys $\rightarrow$ Replace key with predecessor from $child[idx]$, recursively delete predecessor.
 2. Else if $child[idx+1]$ has ≥ 2 keys $\rightarrow$ Replace key with successor from $child[idx+1]$, recursively delete successor.
 3. Else $\rightarrow$ Merge $child[idx]$ and $child[idx+1]$, then delete key from merged node.
 3. Else (key not found in this node):
 - If node is leaf $\rightarrow$ Return (key not present).
 - Else:
 1. Determine $flag = (idx == node.n)$ (if key would be in last child).
 2. If $child[idx]$ has only 1 key:
 - Borrow a key from sibling if possible.
 - Else merge with sibling.
 3. If $flag$ and $idx > node.n$, recurse into $child[idx-1]$, else recurse into $child[idx]$.
-

Algorithm 5: Display (In-order Traversal of B-Tree)

BTree_InOrder(node)

1. For i from 0 to node.n:
 - If node is not leaf → Call BTree_InOrder(node.child[i]).
 - If i < node.n → Print node.keys[i].
2. If node is not leaf → Call BTree_InOrder(node.child[node.n])

CODE :

```
#include <stdio.h>
#include <stdlib.h>

#define ORDER 5
#define MAX ORDER-1
#define MIN ORDER/2-1

struct Node {
    int key[MAX], count, leaf;
    struct Node *child[ORDER];
};

struct Node *root = NULL;

struct Node* create(int leaf) {
    int i;
    struct Node *n = (struct Node*)malloc(sizeof(struct Node));
    n->count = 0; n->leaf = leaf;
    for (i = 0; i < ORDER; i++)
        n->child[i] = NULL;
    return n;
}

void traverse(struct Node *t) {
    int i;
    for (i = 0; i < t->count; i++) {
        if (!t->leaf) traverse(t->child[i]);
        printf("%d ", t->key[i]);
    }
    if (!t->leaf) traverse(t->child[i]);
}
```

```

struct Node* search(struct Node *t, int k) {
    int i = 0;
    while (i < t->count && k > t->key[i]) i++;
    if (i < t->count && t->key[i] == k) return t;
    if (t->leaf) return NULL;
    return search(t->child[i], k);
}

```

```

void split(struct Node *p, int i, struct Node *y) {
    int j;
    struct Node *z = create(y->leaf);

    z->count = MIN;
    for (j = 0; j < MIN; j++)
        z->key[j] = y->key[j + MIN + 1];

    if (!y->leaf) {
        for (j = 0; j <= MIN; j++)
            z->child[j] = y->child[j + MIN + 1];
    }

    y->count = MIN;

    for (j = p->count; j >= i + 1; j--)
        p->child[j + 1] = p->child[j];
    p->child[i + 1] = z;

    for (j = p->count - 1; j >= i; j--)
        p->key[j + 1] = p->key[j];

    p->key[i] = y->key[MIN];
    p->count++;
}

```

```

void insertNonFull(struct Node *x, int k) {
    int i = x->count - 1;
    if (x->leaf) {
        while (i >= 0 && x->key[i] > k) { x->key[i+1] = x->key[i]; i--; }
        x->key[i+1] = k;
        x->count++;
    } else {
        while (i >= 0 && x->key[i] > k) i--;
        i++;
    }
}

```

```

        if (x->child[i]->count == MAX) {
            split(x, i, x->child[i]);
            if (x->key[i] < k) i++;
        }
        insertNonFull(x->child[i], k);
    }
}

```

```

void insert(int k) {
    int i;
    if (!root) {
        root = create(1);
        root->key[0] = k; root->count = 1;
    } else {
        if (root->count == MAX) {
            struct Node *s = create(0);
            s->child[0] = root;
            split(s, 0, root);
            i = 0;
            if (s->key[0] < k) i++;
            insertNonFull(s->child[i], k);
            root = s;
        } else insertNonFull(root, k);
    }
}

```

```

/* ----- Deletion Part (Short Form) ----- */
int findKey(struct Node *x, int k) {
    int idx = 0;
    while (idx < x->count && x->key[idx] < k) ++idx;
    return idx;
}

```

```

int getPred(struct Node *x, int idx) {
    struct Node *cur = x->child[idx];
    while (!cur->leaf) cur = cur->child[cur->count];
    return cur->key[cur->count - 1];
}

```

```

int getSucc(struct Node *x, int idx) {
    struct Node *cur = x->child[idx+1];
    while (!cur->leaf) cur = cur->child[0];
    return cur->key[0];
}

```

```

void merge(struct Node *x, int idx) {
    struct Node *c = x->child[idx], *s = x->child[idx+1];
    int i;
    c->key[MIN] = x->key[idx];
    for (i = 0; i < s->count; i++) c->key[i+MIN+1] = s->key[i];
    if (!c->leaf)
        for (i = 0; i <= s->count; i++) c->child[i+MIN+1] = s->child[i];
    for (i = idx+1; i < x->count; i++) x->key[i-1] = x->key[i];
    for (i = idx+2; i <= x->count; i++) x->child[i-1] = x->child[i];
    c->count += s->count + 1;
    x->count--;
    free(s);
}

```

```

void fill(struct Node *x, int idx) {
    int i;
    if (idx != 0 && x->child[idx-1]->count >= MIN+1) {
        struct Node *c = x->child[idx], *s = x->child[idx-1];
        for (i = c->count-1; i >= 0; i--) c->key[i+1] = c->key[i];
        if (!c->leaf)
            for (i = c->count; i >= 0; i--) c->child[i+1] = c->child[i];
        c->key[0] = x->key[idx-1];
        if (!c->leaf) c->child[0] = s->child[s->count];
        x->key[idx-1] = s->key[s->count-1];
        c->count++; s->count--;
    } else if (idx != x->count && x->child[idx+1]->count >= MIN+1) {
        struct Node *c = x->child[idx], *s = x->child[idx+1];
        c->key[c->count] = x->key[idx];
        if (!c->leaf) c->child[c->count+1] = s->child[0];
        x->key[idx] = s->key[0];
        for (i = 1; i < s->count; i++) s->key[i-1] = s->key[i];
        if (!s->leaf)
            for (i = 1; i <= s->count; i++) s->child[i-1] = s->child[i];
        c->count++; s->count--;
    } else {
        if (idx != x->count) merge(x, idx);
        else merge(x, idx-1);
    }
}

```

```

void deleteKey(struct Node *x, int k) {
    int idx = findKey(x, k), i, pred, flag;
    if (idx < x->count && x->key[idx] == k) {

```

```

    if (x->leaf) {
        for (i = idx+1; i < x->count; i++) x->key[i-1] = x->key[i];
        x->count--;
    } else {
        if (x->child[idx]->count >= MIN+1) {
            pred = getPred(x, idx);
            x->key[idx] = pred;
            deleteKey(x->child[idx], pred);
        } else if (x->child[idx+1]->count >= MIN+1) {
            int succ = getSucc(x, idx);
            x->key[idx] = succ;
            deleteKey(x->child[idx+1], succ);
        } else {
            merge(x, idx);
            deleteKey(x->child[idx], k);
        }
    }
} else {
    if (x->leaf) return;
    flag = (idx == x->count);
    if (x->child[idx]->count < MIN+1) fill(x, idx);
    if (flag && idx > x->count) deleteKey(x->child[idx-1], k);
    else deleteKey(x->child[idx], k);
}
}

```

```

/* ----- Main ----- */

```

```

void main() {
    int arr[] = {15, 3, 9, 20, 6, 1, 25, 30, 12, 18};
    int i, k;
    for (i = 0; i < 10; i++) insert(arr[i]);
    clrscr();
    printf("B-Tree after insertion:\n");
    traverse(root); printf("\n");

    printf("Enter key to search: ");
    scanf("%d", &k);
    if (search(root, k)) printf("Key %d found.\n", k);
    else printf("Key %d not found.\n", k);

    printf("Enter key to delete: ");
    scanf("%d", &k);
    deleteKey(root, k);
    printf("B-Tree after deletion:\n");
}

```

```
    traverse(root); printf("\n");  
    getch();  
}
```

Output :

Experiment 3 :

Problem Statement:

"Construct Min and Max Heap using arrays, delete any element and display the content of the Heap."

Aim

To implement Min Heap and Max Heap using arrays. The program should allow inserting elements, deleting any element, and displaying the heap content after each operation.

Algorithm

Max Heap Construction

1. Start with an array of elements.
2. Insert each element into the heap.
3. For each insertion, "heapify up" to maintain the Max Heap property:
 - Parent node must be greater than its children.

Min Heap Construction

1. Start with an array of elements.
2. Insert each element into the heap.
3. For each insertion, "heapify up" to maintain the Min Heap property:
 - Parent node must be smaller than its children.

Deletion of an Element (from Min/Max Heap)

1. Find the index of the element to delete.
2. Replace it with the last element in the heap.
3. Reduce heap size by 1.
4. Apply heapify (down) to restore heap property.

Display

- Traverse array from index 0 to size-1 and print heap contents.

Sample code:

```
#include <stdio.h>
#define MAX 50
int size;

// Swap function
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Heapify for Max Heap
void heapifyMax(int arr[], int n, int i) {
    int largest = i;
    int left = 2*i + 1;
    int right = 2*i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        heapifyMax(arr, n, largest);
    }
}

// Heapify for Min Heap
void heapifyMin(int arr[], int n, int i) {
    int smallest = i;
    int left = 2*i + 1;
    int right = 2*i + 2;

    if (left < n && arr[left] < arr[smallest])
        smallest = left;

    if (right < n && arr[right] < arr[smallest])
        smallest = right;

    if (smallest != i) {
```

```

        swap(&arr[i], &arr[smallest]);
        heapifyMin(arr, n, smallest);
    }
}

// Build Max Heap
void buildMaxHeap(int arr[], int n) {
    for (int i = n/2 - 1; i >= 0; i--)
        heapifyMax(arr, n, i);
}

// Build Min Heap
void buildMinHeap(int arr[], int n) {
    for (int i = n/2 - 1; i >= 0; i--)
        heapifyMin(arr, n, i);
}

// Delete element from Max Heap
void deleteFromMaxHeap(int arr[], int *n, int value) {
    int i;
    for (i = 0; i < *n; i++) {
        if (arr[i] == value)
            break;
    }

    if (i == *n) {
        printf("Element not found in Max Heap!\n");
        return;
    }

    arr[i] = arr[*n - 1];
    (*n)--;

    buildMaxHeap(arr, *n);
}

// Delete element from Min Heap
void deleteFromMinHeap(int arr[], int *n, int value) {
    int i;
    for (i = 0; i < *n; i++) {
        if (arr[i] == value)
            break;
    }
}

```

```

    if (i == *n) {
        printf("Element not found in Min Heap!\n");
        return;
    }

    arr[i] = arr[*n - 1];
    (*n)--;

    buildMinHeap(arr, *n);
}

// Display Heap
void displayHeap(int arr[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

// Main
int main() {
    int arr[MAX], n, value, i;

    clrscr();
    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    // Max Heap
    buildMaxHeap(arr, n);
    printf("\nMax Heap: ");
    displayHeap(arr, n);

    printf("Enter element to delete from Max Heap: ");
    scanf("%d", &value);
    deleteFromMaxHeap(arr, &n, value);
    printf("Max Heap after deletion: ");
    displayHeap(arr, n);

    // Min Heap
    printf("\nEnter number of elements again: ");
    scanf("%d", &n);

```

```

printf("Enter elements: ");
for (i = 0; i < n; i++)
    scanf("%d", &arr[i]);

buildMinHeap(arr, n);
printf("\nMin Heap: ");
displayHeap(arr, n);

printf("Enter element to delete from Min Heap: ");
scanf("%d", &value);
deleteFromMinHeap(arr, &n, value);
printf("Min Heap after deletion: ");
displayHeap(arr, n);

getch();
}

```

Output :

Enter number of elements: 6
Enter elements: 20 15 30 5 10 25

Max Heap: 30 15 25 5 10 20
Enter element to delete from Max Heap: 25
Max Heap after deletion: 30 15 20 5 10

Enter number of elements again: 6
Enter elements: 20 15 30 5 10 25

Min Heap: 5 10 25 20 15 30
Enter element to delete from Min Heap: 10
Min Heap after deletion: 5 15 25 20 30

Experiments 4 :

Problem Statement:

"Implement BFT (Breadth First Traversal) and DFT (Depth First Traversal) for a given graph, when graph is represented by

a) Adjacency Matrix b) Adjacency List."

Aim

To implement Breadth First Traversal (BFT) and Depth First Traversal (DFT) of a graph using

1. Adjacency Matrix Representation
2. Adjacency List Representation

Algorithm

Breadth First Traversal (BFT)

1. Start with a source vertex.
2. Mark it as visited and enqueue it.
3. While queue is not empty:
 - o Dequeue a vertex.
 - o Visit all its adjacent vertices that are not yet visited, mark them visited, and enqueue them.

Depth First Traversal (DFT)

1. Start with a source vertex.
2. Mark it as visited and print it.
3. Recursively visit all its unvisited adjacent vertices.

Sample code :

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 20

// ----- Queue for BFS -----
int queue[MAX], front = -1, rear = -1;
int visited[MAX];

// Enqueue
void enqueue(int v) {
    if (rear == MAX - 1) return;
    if (front == -1) front = 0;
    queue[++rear] = v;
}

// Dequeue
int dequeue() {
    if (front == -1 || front > rear) return -1;
    return queue[front++];
}

// BFS using adjacency matrix
void BFS_matrix(int adj[MAX][MAX], int n, int start) {
```

```

for (int i = 0; i < n; i++) visited[i] = 0;

enqueue(start);
visited[start] = 1;

printf("BFS (Adjacency Matrix): ");
while (front <= rear) {
    int v = dequeue();
    printf("%d ", v);

    for (int i = 0; i < n; i++) {
        if (adj[v][i] == 1 && !visited[i]) {
            enqueue(i);
            visited[i] = 1;
        }
    }
}
printf("\n");
}

// DFS using adjacency matrix
void DFS_matrix(int adj[MAX][MAX], int n, int v) {
    printf("%d ", v);
    visited[v] = 1;

    for (int i = 0; i < n; i++) {
        if (adj[v][i] == 1 && !visited[i])
            DFS_matrix(adj, n, i);
    }
}

// ----- Adjacency List Representation -----
struct Node {
    int vertex;
    struct Node* next;
};

struct Node* adjList[MAX];

// Add edge for adjacency list
void addEdge(int u, int v) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->vertex = v;
    newNode->next = adjList[u];
}

```

```

    adjList[u] = newNode;
}

// BFS using adjacency list
void BFS_list(int n, int start) {
    for (int i = 0; i < n; i++) visited[i] = 0;
    front = rear = -1;

    enqueue(start);
    visited[start] = 1;

    printf("BFS (Adjacency List): ");
    while (front <= rear) {
        int v = dequeue();
        printf("%d ", v);

        struct Node* temp = adjList[v];
        while (temp) {
            if (!visited[temp->vertex]) {
                enqueue(temp->vertex);
                visited[temp->vertex] = 1;
            }
            temp = temp->next;
        }
    }
    printf("\n");
}

// DFS using adjacency list
void DFS_list(int v) {
    printf("%d ", v);
    visited[v] = 1;

    struct Node* temp = adjList[v];
    while (temp) {
        if (!visited[temp->vertex])
            DFS_list(temp->vertex);
        temp = temp->next;
    }
}

// ----- Main -----
void main() {
    int n, edges, u, v;

```

```

int adj[MAX][MAX];

clrscr();
printf("Enter number of vertices: ");
scanf("%d", &n);

// Initialize adjacency matrix
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
        adj[i][j] = 0;

// Initialize adjacency list
for (int i = 0; i < n; i++)
    adjList[i] = NULL;

printf("Enter number of edges: ");
scanf("%d", &edges);

printf("Enter edges (u v):\n");
for (int i = 0; i < edges; i++) {
    scanf("%d %d", &u, &v);
    adj[u][v] = adj[v][u] = 1; // Undirected graph
    addEdge(u, v);
    addEdge(v, u);
}

int start;
printf("Enter starting vertex: ");
scanf("%d", &start);

// BFS and DFS using adjacency matrix
front = rear = -1;
BFS_matrix(adj, n, start);

for (int i = 0; i < n; i++) visited[i] = 0;
printf("DFS (Adjacency Matrix): ");
DFS_matrix(adj, n, start);
printf("\n");

// BFS and DFS using adjacency list
BFS_list(n, start);

for (int i = 0; i < n; i++) visited[i] = 0;
printf("DFS (Adjacency List): ");

```

```
DFS_list(start);  
printf("\n");  
  
getch();  
}
```

Sample input & output :

Enter number of vertices: 5

Enter number of edges: 6

Enter edges (u v):

0 1

0 2

1 3

1 4

2 3

3 4

Enter starting vertex: 0

BFS (Adjacency Matrix): 0 1 2 3 4

DFS (Adjacency Matrix): 0 1 3 2 4

BFS (Adjacency List): 0 2 1 4 3

DFS (Adjacency List): 0 2 3 4 1